

9 Illustrate the concept of inter-process communication using sharedmemory with a C program

AIM:

To implement the concept of inter-process communication using sharedmemory using C programming.

ALGORITHM:

1. Create a shared memory segment:
 - Use shmget() function to create a new shared memory segment or get the identifier of an existing one.
 - Ensure to handle errors if the shared memory creation fails.
2. Attach shared memory to processes:
 - Use shmat() function to attach the shared memory segment to the process address space.
 - This allows processes to read and write data to the shared memory.
3. Read/Write data in shared memory:
 - Processes can read and write data directly to the shared memory location.
 - Ensure proper synchronization mechanisms (like semaphores) are used to avoid race conditions and maintain data consistency.
4. Detach shared memory and clean up:
 - Use shmdt() function to detach the shared memory segment from the process when done.
 - Optionally, remove the shared memory segment using shmctl() with the IPC_RMID command.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024 // Size of the shared memory segment
int main() {
    key_t key = ftok("shmfile", 65); // Generate a unique key for the shared
    memory segment

    // Create a new shared memory segment (or get the identifier of an existing
    one)
    int shmid = shmget(key, SHM_SIZE, IPC_CREAT | 0666); if (shmid
    == -1) {
        perror("shmget");
        exit(EXIT_FAILURE);
    }
```

```

}

// Attach the shared memory segment to the process address space
char *shm_ptr = (char*)shmat(shmid, NULL, 0);if
(shm_ptr == (char*)(-1)) {
    perror("shmat");
    exit(EXIT_FAILURE);
}

// Write data to the shared memory strcpy(shm_ptr,
"Hello, shared memory!");

// Detach the shared memory segment from the processif
(shmdt(shm_ptr) == -1) {
    perror("shmdt");
    exit(EXIT_FAILURE);
}

printf("Data written to shared memory: %s\n", shm_ptr);

// Optional: Remove the shared memory segmentif
(shmctl(shmid, IPC_RMID, NULL) == -1) {
    perror("shmctl");
    exit(EXIT_FAILURE);
}

return 0;
}

```

OUTPUT:

vbnet

Data written to shared memory: Hello, shared memory!

10. Illustrate the concept of inter-process communication using message queue with a c program

AIM :

To implement the concept of inter-process communication using message queue with a c program

ALGORITHM :

1. Create a message queue:
 - i. Use msgget() function to create a new message queue or get the identifier of an existing one.
 - ii. Ensure to handle errors if the message queue creation fails.
2. Send a message to the queue (Producer process):
 - i. Define a structure for the message containing necessary data fields.
 - ii. Populate the message structure with appropriate data.
 - iii. Use msgsnd() function to send the message to the message queue.
 - iv. Handle errors if message sending fails.
3. Receive a message from the queue (Consumer process):
 - i. Define a structure for the message to receive data.
 - ii. Use msgrcv() function to receive a message from the message queue.
 - iii. Process the received message as needed.
 - iv. Handle errors if message receiving fails.
4. Remove the message queue (Optional):
 - i. Use msgctl() function with IPC_RMID command to remove the message queue when it's no longer needed.

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct message {
    long msg_type;
    char msg_text[100];
};

int main() {
    key_t key = ftok("msgqfile", 65); // Generate a unique key for the message queue

    // Create a new message queue (or get the identifier of an existing one)
    int msgid = msgget(key, IPC_CREAT | 0666);
    if (msgid == -1) {
        perror("msgget");
    }
```

```

        exit(EXIT_FAILURE);
    }

    struct message msg;
    msg.msg_type = 1; // Message type (can be any positive number)

    // Producer: Send a message to the message queue
    strcpy(msg.msg_text, "Hello, message queue!");
    if (msgsnd(msgid, (void*)&msg, sizeof(msg.msg_text),
IPC_NOWAIT) == -1) {
        perror("msgsnd");
        exit(EXIT_FAILURE);
    }

    printf("Producer: Data sent to message queue: %s\n", msg.msg_text);

    // Consumer: Receive a message from the message queue
    if (msgrcv(msgid, (void*)&msg, sizeof(msg.msg_text), 1, 0) == -1) {
        perror("msgrcv");
        exit(EXIT_FAILURE);
    }

    printf("Consumer: Data received from message queue: %s\n",
msg.msg_text);

    // Remove the message queue
    if (msgctl(msgid, IPC_RMID, NULL) == -1) {
        perror("msgctl");
        exit(EXIT_FAILURE);
    }

    return 0;
}

```

OUTPUT :



```

arduino
Producer: Data sent to message queue: Hello, message queue!
Consumer: Data received from message queue: Hello, message queue!

```

11. Illustrate the concept of multithreading using a C program

AIM :

To implement the concept of multithreading using C program

ALGORITHM :

Include Necessary Libraries:

Include the necessary header files for multithreading. For POSIX threads in C, include pthread.h.

1. Define Thread Function:
 - i. Define a function that will be executed by the threads. This function should have a void* return type and a void* parameter, which can be used to pass data to the thread.
2. Create Threads:
 - i. In the main function or any other function, create thread objects using pthread_t.
 - ii. Use pthread_create() to create threads and specify the thread function as well as any parameters to pass.
3. Thread Execution:
 - i. Threads execute concurrently and perform the tasks defined in the thread function.
4. Synchronization and Coordination (Optional):
 - i. Use synchronization mechanisms such as mutexes, semaphores, or condition variables to coordinate the execution of threads and ensure data consistency.
5. Wait for Threads to Complete (Optional):
 - i. Use pthread_join() to wait for threads to finish their execution if the main thread needs to synchronize with the created threads.
 - ii. Thread Termination (Optional):
 - iii. Threads can exit by returning from the thread function or by calling pthread_exit(). The main thread can also call pthread_exit() to terminate the entire process.

PROGRAM :

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
void* threadFunction(void* arg) {char*  
    message = (char*)arg; printf("%s\n",  
    message);
```

```
    return NULL;
}

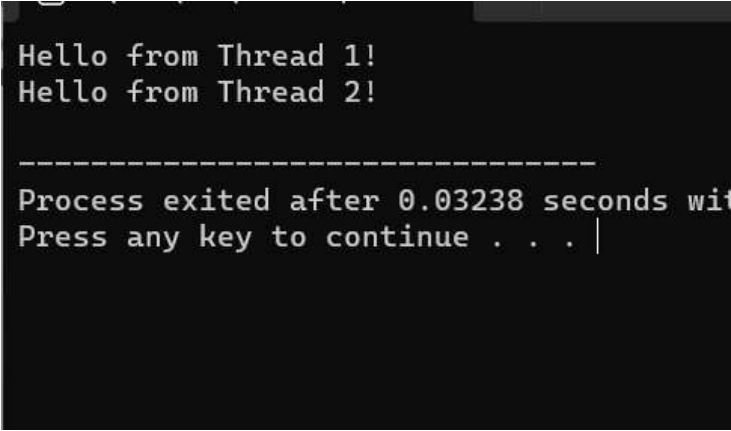
int main() {
    pthread_t thread1, thread2;
    char* message1 = "Hello from Thread 1!";char*
    message2 = "Hello from Thread 2!";

    // Create threads
    pthread_create(&thread1, NULL, threadFunction, (void*)message1);
    pthread_create(&thread2, NULL, threadFunction, (void*)message2);

    // Wait for threads to complete
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}
```

OUTPUT :



```
Hello from Thread 1!
Hello from Thread 2!

-----
Process exited after 0.03238 seconds with
Press any key to continue . . . |
```

12. Design a C program to simulate the concept of Dining-Philosophers problem

AIM :

To design a C program to simulate the concept of Dining-Philosophers problem

ALGORITHM :

1. Initialize Semaphores:
 - i. Create an array of semaphores, one for each philosopher, to control access to the chopsticks.
 - ii. Initialize each semaphore to 1, indicating that the chopstick is available.
2. Define Philosopher Structure:
 - i. Define a structure to represent a philosopher, which includes their ID and the semaphores representing the left and right chopsticks.
3. Philosopher Lifecycle:
 - i. Each philosopher runs as a separate thread. In the thread function:
 - ii. Think: Philosopher thinks for a random amount of time.
 - iii. Pick up Chopsticks: Philosopher tries to pick up the left and right chopsticks (acquire semaphores). If both are available, the philosopher proceeds to eat. If not, they release the acquired chopstick(s) and return to thinking.
 - iv. Eat: Philosopher eats for a random amount of time.
 - v. Put Down Chopsticks: Philosopher releases the left and right chopsticks (release semaphores) after eating.
 - vi. Repeat the above steps indefinitely to simulate the philosopher's life cycle.
4. Implement Deadlock Avoidance:
 - i. To avoid deadlock, impose a constraint such that a philosopher can only pick up both chopsticks if both are available.
 - ii. One way to achieve this is by introducing a global semaphore that limits the number of philosophers allowed to pick up chopsticks simultaneously. For example, if there are 5 philosophers, allow at most 4 philosophers to pick up chopsticks simultaneously. This prevents the circular wait condition and avoids deadlock.
5. Compile and Run:
 - i. Compile the program with appropriate multithreading flags (e.g., -pthread for gcc on Unix-based systems).
 - ii. Run the program to observe the dining philosophers problem simulation.

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define NUM_PHILOSOPHERS 5
```

```

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void* philosopherLifeCycle(void* arg) {int id =
    *((int*)arg);
    int left_chopstick = id;
    int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;

    while (1) {
        // Think
        printf("Philosopher %d is thinking...\n", id);

        // Pick up chopsticks
        pthread_mutex_lock(&chopsticks[left_chopstick]);
        pthread_mutex_lock(&chopsticks[right_chopstick]);

        // Eat
        printf("Philosopher %d is eating...\n", id);
        sleep(rand() % 3 + 1); // Eating time

        // Put down chopsticks
        pthread_mutex_unlock(&chopsticks[left_chopstick]);
        pthread_mutex_unlock(&chopsticks[right_chopstick]);

        // Repeat the cycle
    }
}

int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];int
    philosopher_ids[NUM_PHILOSOPHERS];

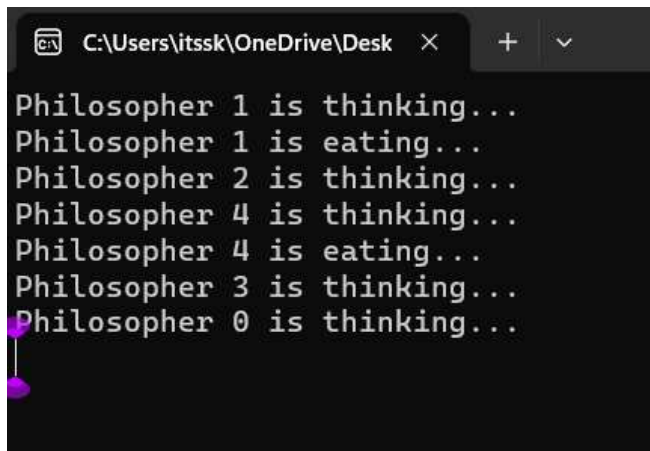
    // Initialize mutex locks
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        pthread_mutex_init(&chopsticks[i], NULL);
    }

    // Create philosopher threads
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        philosopher_ids[i] = i;

```

```
        pthread_create(&philosophers[i], NULL, philosopherLifeCycle,  
(void*)&philosopher_ids[i]);  
    }  
  
    // Wait for threads to finish (although they run indefinitely)for (int i =  
    0; i < NUM_PHILOSOPHERS; ++i) {  
        pthread_join(philosophers[i], NULL);  
    }  
  
    // Destroy mutex locks  
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {  
        pthread_mutex_destroy(&chopsticks[i]);  
    }  
  
    return 0;  
}
```

OUTPUT :



```
C:\Users\itssk\OneDrive\Desk  
Philosopher 1 is thinking...  
Philosopher 1 is eating...  
Philosopher 2 is thinking...  
Philosopher 4 is thinking...  
Philosopher 4 is eating...  
Philosopher 3 is thinking...  
Philosopher 0 is thinking...
```

13. Construct a C program to implement various memory allocation strategies.

AIM :

To construct a C program to implement various memory allocation strategies.

ALGORITHM :

1. Include Necessary Libraries:
Include the necessary header files such as `stdio.h`, `stdlib.h`, etc.
2. Define Process Control Block (PCB) Structure:
 - i. Define a structure to represent a Process Control Block (PCB) that
 - ii. contains information about each process, including process ID,
 - iii. memory size, and allocation status.
3. Implement Memory Allocation Functions:
 - i. Implement functions for memory allocation strategies like First Fit, Best Fit, and Worst Fit.
 - ii. Each function should search for a suitable block of memory in the memory pool based on the specific strategy (first fit, best fit, or worst fit).
 - iii. Allocate memory to the process by updating the allocation status in the PCB and updating the memory pool accordingly.
4. Implement Memory Deallocation Function:
 - i. Implement a function to deallocate memory occupied by a process.
 - ii. Update the allocation status in the PCB and release the memory block, merging it with adjacent free blocks if necessary.
5. Main Function:
 - i. In the main function, initialize the memory pool (an array representing the available memory).
 - ii. Create PCBs for processes with specific memory requirements.
 - iii. Call the appropriate memory allocation functions based on the desired strategy for each process.
 - iv. Deallocate memory for completed processes using the memory deallocation function.
6. Print Memory Allocation Status:
 - i. Implement a function to print the memory allocation status after

each allocation and deallocation operation.

7. Compile and Run:
 - i. Compile the program and run it to observe how different memory allocation strategies work.

PROGRAM :

```
#include<stdio.h>

void bestfit(int mp[],int p[],int m,int n){int
    j=0;
    for(int i=0;i<n;i++){
        if(mp[i]>p[j]){
            printf("\n%d fits in %d",p[j],mp[i]);
            mp[i]=mp[i]-p[j++];
            i=i-1;
        }
    }
    for(int i=j;i<m;i++)
    {
        printf("\n%d must wait for its process",p[i]);
    }
}

void rsort(int a[],int n){ for(int
    i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(a[i]>a[j]){
                int t=a[i];
                a[i]=a[j];
                a[j]=t;
            }
        }
    }
}
```

```
}
```

```
void sort(int a[],int n){ for(int
```

```
    i=0;i<n;i++){
```

```
        for(int j=0;j<n;j++){
```

```
            if(a[i]<a[j]){
```

```
                int t=a[i];
```

```
                a[i]=a[j];
```

```
                a[j]=t;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void firstfit(int mp[],int p[],int m,int n){
```

```
    sort(mp,n);
```

```
    sort(p,m);
```

```
    bestfit(mp,p,m,n);
```

```
}
```

```
void worstfit(int mp[],int p[],int m,int n){
```

```
    rsort(mp,n);
```

```
    sort(p,m);
```

```
    bestfit(mp,p,m,n);
```

```
}
```

```
int main(){
```

```
    int m,n,mp[20],p[20],ch; printf("Number of
```

```
memory partition : ");scanf("%d",&n);
```

```
    printf("Number of process : ");
```

```
    scanf("%d",&m);
```

```
    printf("Enter the memory partitions : \n");for(int
```

```
    i=0;i<n;i++){
```

```
        scanf("%d",&mp[i]);
```

```
    }
```

```
    printf("ENter process size : \n");
```

```
    for(int i=0;i<m;i++){
```

```

        scanf("%d",&p[i]);
    }
    printf("1. Firstfit\t2. Bestfit\t3. worstfit\nEnter your choice : ");
    scanf("%d",&ch);
    switch(ch){
    case 1:
        bestfit(mp,p,m,n);
        break;
    case 2:
        firstfit(mp,p,m,n);
        break;
    case 3:
        worstfit(mp,p,m,n);
        break;
    default:
        printf("invalid");
        break;
    }
}

```

OUTPUT :

```
C:\Users\itssk\OneDrive\Desktop >
Number of memory partition : 5
Number of process : 4
Enter the memory partitions :
150
220
500
350
700
Enter process size :
160
450
500
412
1. Firstfit      2. Bestfit      3. worstfit
Enter your choice : 1

160 fits in 220
450 fits in 500
500 fits in 700
412 must wait for its process
-----
Process exited after 31.7 seconds with return
Press any key to continue . . .
```

14. Construct a C program to organize the file using single level directory

AIM:

To construct a c program to organize the file using single level directory

ALGORITHM :

Step 1: Define Structures

Define structures to represent files and the directory.

Step 2: Initialize Directory

Create a function or code segment to initialize the directory structure. Set the initial file count to 0.

Step 3: Add Files

Implement a function or code segment to add files to the directory. This function should handle adding files, updating the file count, and handling errors if the directory is full.

Step 4: List Files

Create a function or code segment to list all the files in the directory. This function should iterate through the file list and print the file names.

Step 5: Delete Files (Optional)

Implement a function or code segment to delete files from the directory. This function should handle removing files, updating the file count, and handling errors if the file is not found.

Step 6: Implement User Interface

Create a user interface for interacting with the program. This could be a menu-driven interface where users can choose to add files, list files, delete files, or exit the program.

Step 7: Test the Program

Compile the program using a C compiler and test it by adding files, listing files, and deleting files. Make sure the program handles different scenarios and errors gracefully.

Step 8: Refine and Expand (Optional)

Refine your program based on testing results. You can also expand the functionality by adding more features, error handling, or optimizing the code.

Step 9: Document Your Code (Optional)

Document your code by adding comments to explain the functionality of different sections of your program. This will make it easier for others (and yourself) to understand the code in the future.

Step 10: Compile and Distribute

Once your program is complete and thoroughly tested, compile it into an executable file. If you want to distribute the program, you can create an installer or provide the

executable along with necessary instructions.

PROGRAM :

```
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>

#define BUFFER_SIZE 4096

void copy() {
    const char *sourcefile=
"C:/Users/itssk/OneDrive/Desktop/sasi.txt";

    const char *destination_file="C:/Users/itssk/OneDrive/Desktop/sk.txt";
    int source_fd = open(sourcefile, O_RDONLY);
    int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC,
0666);

    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    while ((bytesRead = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
        bytesWritten = write(dest_fd, buffer, bytesRead);
    }

    close(source_fd);
    close(dest_fd);

    printf("File copied successfully.\n");
}

void create()
{
    char path[100];

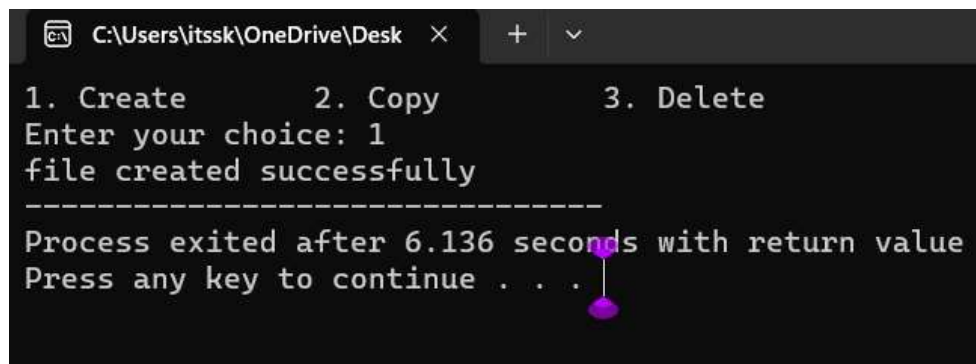
    FILE *fp;
    fp=fopen("C:/Users/itssk/OneDrive/Desktop/sasi.txt","w");
    printf("file created successfully");
}
```

```
int main(){
    int n;

    printf("1. Create \t2. Copy \t3. Delete\nEnter your choice: " );
    scanf("%d",&n);
    switch(n){
        case 1:
            create(); break;
        case 2:
            copy();
            break;
        case 3:
            remove("C:/Users/itssk/OneDrive/Desktop/sasi.txt");printf("Deleted
            successfully");

    }}
}
```

OUTPUT :



```
C:\Users\itssk\OneDrive\Desktop >
1. Create      2. Copy      3. Delete
Enter your choice: 1
file created successfully
-----
Process exited after 6.136 seconds with return value
Press any key to continue . . .
```

15.Design a C program to organize the file using two level directorystructure.

AIM :

To design a C program to organize the file using two level directorystructure

Algorithm :

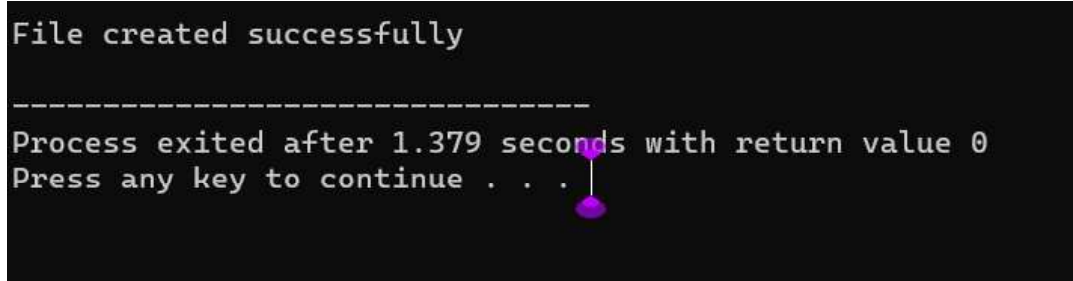
1. Define Structures: Define structures for files and directories. Each directory structure should contain an array for files and an array for subdirectories.
2. Initialize Root Directory: Create a root directory structure. This serves as the starting point for the two-level directory structure.
3. Add Files to Directories: Implement a function to add files to a specific directory. Handle adding files, updating the file count, and handling errors if the directory is full.
4. Add Subdirectories: Implement a function to add subdirectories to a specific directory. Manage adding directories, updating the directory count, and handling errors if the parent directory is full.
5. List Files and Subdirectories: Create functions to list all the files and subdirectories in a directory. These functions should iterate through the file and subdirectory arrays and print their names.
6. Delete Files and Subdirectories (Optional): Implement functions to delete files and subdirectories from a directory. Handle removing files or directories, updating the counts, and handling errors if the file or directory is not found.
7. Implement User Interface: Design a user interface for interacting with the program. This could be a menu-driven interface where users can add files, add subdirectories, list files, list subdirectories, delete files, delete subdirectories, or exit the program.
8. Test the Program: Compile the program and test it thoroughly. Add files, add subdirectories, list files, list subdirectories, delete files, and delete subdirectories. Ensure the program handles different scenarios and errors gracefully.
9. Refine and Expand (Optional): Refine the program based on testing results. Expand the functionality by adding more features, error handling, or optimizing the code.
10. Document Your Code (Optional): Document your code by adding comments to explain the functionality of different sections. This will make it easier for others to understand the code in the future.

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int
main() {
    char mainDirectory[] = "C:/Users/itssk/OneDrive/Desktop";char
    subDirectory[] = "os";
    char fileName[] = "example.txt";char
    filePath[200];
    char mainDirPath[200];
    snprintf(mainDirPath, sizeof(mainDirPath), "%s/%s/", mainDirectory, subDirectory);
```

```
snprintf(filePath, sizeof(filePath), "%s%s", mainDirPath, fileName); FILE *file
= fopen(filePath, "w");
if (file == NULL) { printf("Error
    creating file.\n"); return 1;
}
fprintf(file, "This is an example file content.");
printf("File created successfully: %s\n");
}
```

OUTPUT :

A terminal window with a black background and white text. The output shows 'File created successfully' followed by a dashed line separator. Below the separator, it says 'Process exited after 1.379 seconds with return value 0' and 'Press any key to continue . . .'. A blue cursor is positioned at the end of the last line.

```
File created successfully
-----
Process exited after 1.379 seconds with return value 0
Press any key to continue . . .
```

16. Develop a C program for implementing random access file for processing the employee details

AIM :

To develop a C program for implementing random access file for processing the employee details

ALGORITHM :

1. **Define Structure:** Define a structure to represent employee details. Include attributes like employee ID, name, salary, and any other relevant information.
2. **Open File in Binary Mode:** Open a file in binary mode using the `fopen` function. Specify the file path and mode ("rb+" for reading and writing binary files).
3. **Menu-Driven Interface:** Create a menu-driven interface for the user to perform operations. Options could include adding a new employee, updating existing employee details, searching for an employee, deleting an employee, listing all employees, and exiting the program.
4. **Implement Functions:** Implement functions corresponding to each menu option. For example, implement functions to add a new employee, update employee details, search for an employee by ID, delete an employee, and list all employees. These functions should perform file operations like reading and writing records.
5. **Random Access File Operations:** Utilize `fseek` and `ftell` functions to perform random access file operations. Use `fseek` to move the file pointer to the desired record based on the employee ID and `ftell` to determine the current position of the file pointer.
6. **File Read and Write:** Implement functions to read and write employee records to the file. Use `fread` and `fwrite` functions to read and write structures to the file.
7. **Error Handling:** Implement error handling to deal with situations where the file cannot be opened or when operations like adding, updating, or deleting employees fail. Display appropriate error messages to the user.
8. **Close the File:** Close the file using the `fclose` function when the program is exiting or when the file operations are completed.
9. **Testing:** Test the program thoroughly by adding, updating, searching, and deleting employee records. Ensure that the program handles edge cases and errors gracefully.
10. **Documentation (Optional):** Add comments and documentation to your code to explain the functionality of different sections, making it easier for others (and yourself) to understand the code in the future.

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>

struct Employee {
```

```

int empId;

char empName[50];

float empSalary;};

int main() { FILE
    *filePtr;

    struct Employee emp;

    filePtr = fopen("employee.dat", "rb+");if
    (filePtr == NULL) {
        filePtr = fopen("employee.dat", "wb+");if
        (filePtr == NULL) {
            printf("Error creating the file.\n");
            return 1;    }
        }

    int choice;

    do {
        printf("\nEmployee    Database    Menu:\n");
        printf("1. Add Employee\n");
        printf("2.  Display  Employee  Details\n");
        printf("3.   Update   Employee   Details\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice); switch
        (choice) {
            case 1:
                printf("Enter Employee ID: "); scanf("%d",
                    &emp.empId); printf("Enter Employee
                    Name: ");

```

```

scanf("%s", emp.empName);
printf("Enter Employee Salary: ");
scanf("%f", &emp.empSalary);
fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee),
SEEK_SET);

fwrite(&emp, sizeof(struct Employee), 1, filePtr);
printf("Employee details added successfully.\n");break;

case 2:

printf("Enter Employee ID to display: ");
scanf("%d", &emp.empId);
fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee),
SEEK_SET);

fread(&emp, sizeof(struct Employee), 1, filePtr);
printf("Employee ID: %d\n", emp.empId); printf("Employee
Name: %s\n", emp.empName); printf("Employee Salary:
%.2f\n", emp.empSalary);break;

case 3:

printf("Enter Employee ID to update: ");
scanf("%d", &emp.empId);
fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee),
SEEK_SET);

fread(&emp, sizeof(struct Employee), 1, filePtr);
printf("Enter Employee Name: ");
scanf("%s", emp.empName);
printf("Enter Employee Salary: ");
scanf("%f", &emp.empSalary);

```

```

        fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee),
SEEK_SET);

        fwrite(&emp, sizeof(struct Employee), 1, filePtr);

        printf("Employee details updated successfully.\n");break;

case 4:

    break;

default:

    printf("Invalid choice. Please try again.\n");

    }

} while (choice != 4);

fclose(filePtr);

return 0;

```

OUTPUT :

```

C:\Users\itssk\OneDrive\Desk
Employee Database Menu:
1. Add Employee
2. Display Employee Details
3. Update Employee Details
4. Exit
Enter your choice: 1
Enter Employee ID: 567
Enter Employee Name: sasi
Enter Employee Salary: 50000
Employee details added successfully.

Employee Database Menu:
1. Add Employee
2. Display Employee Details
3. Update Employee Details
4. Exit
Enter your choice: |

```

17. Illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

AIM :

To illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

ALGORITHM :

1. **Define Data Structures:** Define appropriate data structures to store the available resources, maximum resources, allocated resources, and need matrix for each process.
2. **Initialize Matrices:** Initialize the available, maximum, allocated, and need matrices based on the system's resources and the maximum demand of each process.
3. **Input Request:** Implement a function to input resource request from processes. This function should validate if the request is within the maximum limit specified by each process.
4. **Safety Algorithm:** Implement the Banker's safety algorithm to check if the system is in a safe state. Use the available, allocated, and need matrices to determine if the system can allocate resources to processes without entering into a deadlock state.
5. **Resource Allocation:** Implement resource allocation functions to handle the request from processes. Check if the request can be granted safely using the Banker's algorithm. If the request can be granted, update the allocated and available matrices accordingly.
6. **User Interface:** Create a user interface to interact with the program. Allow users to input resource requests and display the current state of the system, including available resources and resource allocation status.
7. **Deadlock Scenario:** Introduce scenarios where a deadlock can potentially occur (e.g., requesting more resources than available or requesting resources in a circular wait condition).
8. **Testing:** Test the program with different resource request scenarios. Ensure that the system handles requests properly without entering into a deadlock state. Also, test scenarios where the system should deny requests to prevent deadlock.

9. Documentation (Optional): Add comments and documentation to the code to explain the functionality of different sections. Document the Banker's algorithm steps and how it prevents deadlocks in the system.

10. Error Handling: Implement error handling mechanisms to deal with invalid input, unexpected scenarios, or any issues that might occur during resource allocation and deallocation.

PROGRAM :

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3int
is_safe();
int available[MAX_RESOURCES] = {3, 3, 2}; // Available instances of each resource
int maximum[MAX_PROCESSES][MAX_RESOURCES] = {{7, 5, 3},
{3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int allocation[MAX_PROCESSES][MAX_RESOURCES] = {{0, 1, 0},
{2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

int request_resources(int process_num, int request[]) {
    // Check if request can be granted
    for (int i = 0; i < MAX_RESOURCES; i++) {
        if (request[i] > available[i] || request[i] > maximum[process_num][i]
- allocation[process_num][i])
            return 0; // Request cannot be granted
    }

    // Try allocating resources temporarily
    for (int i = 0; i < MAX_RESOURCES; i++) {
        available[i] -= request[i];
```

```

        allocation[process_num][i] += request[i];

        // Update maximum and need matrix if request is granted
        maximum[process_num][i] -= request[i];
    }

    // Check if system is in safe state after allocation
    if (is_safe()) {
        return 1; // Request is granted
    } else {
        // Roll back changes if not safe
        for (int i = 0; i < MAX_RESOURCES; i++) {
            available[i] += request[i];
            allocation[process_num][i] -= request[i];
            maximum[process_num][i] += request[i];
        }
        return 0; // Request is denied
    }
}

```

```

int is_safe() {
    int work[MAX_RESOURCES];
    int finish[MAX_PROCESSES] = {0};

    // Initialize work array
    for (int i = 0; i < MAX_RESOURCES; i++) {work[i]
        = available[i];
    }
}

```

```

// Check if processes can finish
count = 0;
while (count < MAX_PROCESSES) {
    int found = 0;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (finish[i] == 0) {
            int j;
            for (j = 0; j < MAX_RESOURCES; j++) {
                if (maximum[i][j] - allocation[i][j] > work[j]) break;
            }
            if (j == MAX_RESOURCES) {
                // Process can finish, update work and mark as finished
                for (int k = 0; k < MAX_RESOURCES; k++) {
                    work[k] += allocation[i][k];
                }
                finish[i] = 1;
                found = 1;
                count++;
            }
        }
    }
    if (found == 0) {
        return 0; // No process can finish, not safe state
    }
}

```

```

        return 1; // All processes can finish, safe state
    }

int main() {
    int process_num, request[MAX_RESOURCES];
    printf("Enter process number (0 to 4): "); scanf("%d",
    &process_num);

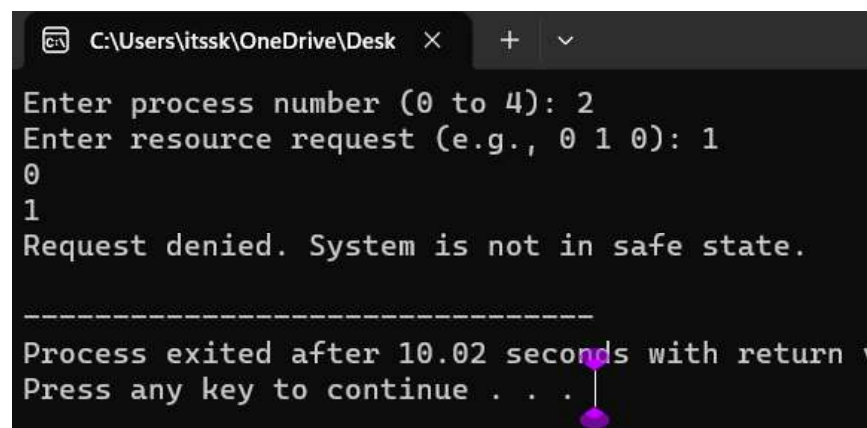
    printf("Enter resource request (e.g., 0 1 0): ");for (int
    i = 0; i < MAX_RESOURCES; i++) {
        scanf("%d", &request[i]);
    }

    if (request_resources(process_num, request)) {
        printf("Request granted.\n");
    } else {
        printf("Request denied. System is not in safe state.\n");
    }

    return 0;
}

```

OUTPUT :



```

C:\Users\itssk\OneDrive\Desktop
Enter process number (0 to 4): 2
Enter resource request (e.g., 0 1 0): 1
0
1
Request denied. System is not in safe state.

-----
Process exited after 10.02 seconds with return v
Press any key to continue . . .

```

18. Construct a C program to simulate producer consumer problem using semaphores.

AIM :

To construct a C program to simulate producer consumer problem using semaphores.

ALGORITHM :

1. **Include Libraries:** Include necessary libraries such as `stdio.h`, `stdlib.h`, `pthread.h` for threads, and `semaphore.h` for semaphores.
2. **Define Constants:** Define constants such as the size of the buffer (maximum number of items), number of producer and consumer threads, etc.
3. **Declare Global Variables:** Declare global variables including the buffer (an array to hold the items), indices to track the next position for inserting and removing items, and semaphores for synchronization.
4. **Initialize Semaphores:** Initialize semaphores for controlling access to the buffer, tracking empty spaces in the buffer, and tracking available items in the buffer.
5. **Create Producer and Consumer Threads:** Create threads for producers and consumers. Each thread should have its own unique identifier (for example, an integer value).
6. **Define Producer and Consumer Functions:** Implement functions for producers and consumers. These functions will be executed by the corresponding threads. The producer function will generate items and insert them into the buffer, while the consumer function will remove items from the buffer.
7. **Implement Buffer Operations:** Implement functions for inserting items into the buffer (enqueue operation) and removing items from the buffer (dequeue operation). Use semaphores to control access to the buffer and update the indices accordingly.
8. **Synchronize Producer and Consumer Threads:** Use semaphores to synchronize the producer and consumer threads. The producer should wait if the buffer is full, and the consumer should wait if the buffer is empty.

9. Handle Thread Joining and Cleanup: After creating the threads, ensure that the main program waits for all threads to finish their execution. Use `pthread_join` for this purpose. Also, clean up any resources allocated during the program execution.

10. Compile and Run: Compile the C program using a C compiler (such as `gcc`) and run the executable. Observe the behavior of the producer and consumer threads, ensuring that they are properly synchronized and the buffer operations are correctly implemented.

PROGRAM :

```
#include <stdio.h> #include
<pthread.h> #include
<semaphore.h>
#include<Windows.h>

#define BUFFER_SIZE 5

#define MAX_ITEMS 10 // Maximum number of items to be
produced/consumed

int buffer[BUFFER_SIZE];
sem_t empty, full;
int produced_items = 0, consumed_items = 0;

void* producer(void* arg) {
    while (produced_items < MAX_ITEMS) {
        sem_wait(&empty);
        // Critical section: add item to buffer
        for (int i = 0; i < BUFFER_SIZE; ++i) {if
            (buffer[i] == 0) {
                buffer[i] = produced_items + 1;
                printf("Produced: %d\n", buffer[i]);
```

```

        produced_items++;
        break;
    }
}
sem_post(&full);
Sleep(1); // Sleep for a while
}
return NULL;
}

void* consumer(void* arg) {
    while (consumed_items < MAX_ITEMS) {
        sem_wait(&full);
        // Critical section: remove item from bufferfor (int
        i = 0; i < BUFFER_SIZE; ++i) {
            if (buffer[i] != 0) {
                printf("Consumed: %d\n", buffer[i]);
                buffer[i] = 0;
                consumed_items++;
                break;
            }
        }
        sem_post(&empty); Sleep(2); //
        Sleep for a while
    }
    return NULL;
}

```

```
int main() {  
    pthread_t producer_thread, consumer_thread;  
  
    sem_init(&empty, 0, BUFFER_SIZE);  
    sem_init(&full, 0, 0);  
  
    // Create producer and consumer threads pthread_create(&producer_thread,  
    NULL, producer, NULL); pthread_create(&consumer_thread, NULL,  
    consumer, NULL);  
  
    // Wait for threads to finish  
    pthread_join(producer_thread, NULL);  
    pthread_join(consumer_thread, NULL);  
  
    // Destroy semaphores  
    sem_destroy(&empty);  
    sem_destroy(&full);  
  
    return 0;  
}
```

OUTPUT :

C:\Users\itssk\OneDrive\Desk

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
```

19. Design a C program to implement process synchronization using mutex locks.

AIM:

To design a C program to implement process synchronization using mutex locks.

ALGORITHM :

Step 1: Include Necessary Libraries: Include the required header files for pthreads and mutex locks.

Step 2: Declare Global Variables: Declare any global variables needed for synchronization, such as mutex variables.

Step 3: Initialize Mutex: In the main function or initialization function, initialize the mutex using pthread_mutex_init function.

Step 4: Define Functions: Define functions that represent the actions of threads. These functions should include the critical sections where the mutex lock is acquired and released.

Step 5: Create Threads: In the main function or any other appropriate function, create threads and assign the functions to execute for each thread. Pass NULL or any necessary data as arguments to the functions.

Step 6: Implement Mutex Synchronization: Inside the functions that represent the actions of threads, use pthread_mutex_lock to acquire the mutex lock and pthread_mutex_unlock to release the lock. This ensures that only one thread can execute the critical section at a time.

Step 7: Join Threads and Cleanup: In the main function or any other appropriate function, wait for the threads to finish using pthread_join. After the threads have finished their execution, destroy the mutex using pthread_mutex_destroy function.

Step 8: Compile and Run: Compile the program using a C compiler with the appropriate flags (for example, -pthread for GCC) to link the pthread library. Then, run the compiled executable to observe the synchronized behavior of threads due to mutex locks.

PROGRAM :

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
// Shared variables
```

```
int
```

```
counter = 0;
```

```
pthread_mutex_t mutex;
```

```
// Function to be executed by threads
```

```
void *threadFunction(void *arg) {
```

```
int i;  
for (i = 0; i < 1000000; ++i) {  
    return NULL;  
}
```

```
int main() {
```

```
pthread_mutex_init(&mutex, NULL);
pthread_t thread1, thread2;
pthread_create(&thread1, NULL, threadFunction, NULL);
pthread_create(&thread2, NULL, threadFunction, NULL);

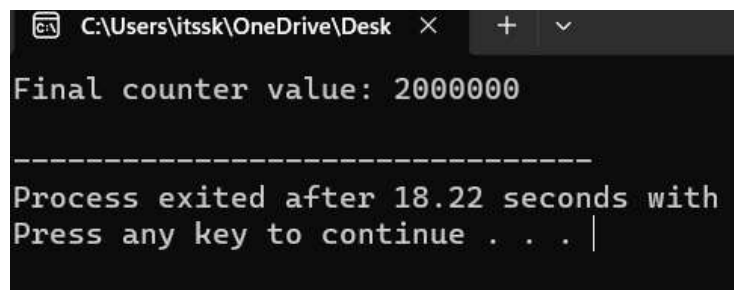
// Wait for the threads to finish
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// Destroy the mutex
pthread_mutex_destroy(&mutex);

// Print the final value of the counter printf("Final
counter value: %d\n", counter);

return 0;
}
```

OUTPUT :



```
C:\Users\itssk\OneDrive\Desktop  X  +  v
Final counter value: 2000000
-----
Process exited after 18.22 seconds with
Press any key to continue . . . |
```

20. Construct a C program to simulate Reader-Writer problem using semaphores

AIM :

To construct a C program to simulate Reader-Writer problem using semaphores

ALGORITHM :

1. Include Libraries: Include necessary libraries for using semaphores, threads, and other required functionalities.
2. Initialize Semaphores: Create semaphores to control access to the shared resources:
 - Semaphore for Readers Count: Initialize a semaphore to 1 (binary semaphore).
 - Semaphore for Writers Count: Initialize a semaphore to 1 (binary semaphore).
 - Semaphore for Readers Waiting: Initialize a semaphore to 1 (binary semaphore).
 - Semaphore for Writers Waiting: Initialize a semaphore to 1 (binary semaphore).
 - Semaphore for Mutex: Initialize a semaphore to 1 (binary semaphore).
3. Reader Function: Create a function for readers to execute. This function should handle the logic for readers accessing the shared resource.
4. Writer Function: Create a function for writers to execute. This function should handle the logic for writers accessing the shared resource.
5. Implement Reader-Writer Logic: Inside the reader and writer functions, implement the logic that ensures proper synchronization using semaphores. Readers should check and update the readers count semaphore and writers should check and update the writers count semaphore.
6. Create Threads: In your main function, create multiple threads for readers and writers to simulate concurrent access.
7. Join Threads: Use thread joining functions to wait for all threads to complete their execution.
8. Clean Up: Destroy the semaphores and perform any necessary clean-up operations before exiting the program.

PROGRAM :

```
#include <stdio.h> #include
<pthread.h> #include
<semaphore.h>

sem_t mutex, writeBlock;
int data = 0, readersCount = 0;

void *reader(void *arg) {int
    i=0;
    while (i<10) {
        sem_wait(&mutex);
        readersCount++;
        if (readersCount == 1) {
            sem_wait(&writeBlock);
        }
        sem_post(&mutex);

        // Reading operation
        printf("Reader reads data: %d\n", data);

        sem_wait(&mutex);
        readersCount--;
        if (readersCount == 0) {
            sem_post(&writeBlock);
        }
        sem_post(&mutex);
```

```
        i++;  
    }  
}
```

```
void *writer(void *arg) {int  
    i=0;  
    while (i<10) {  
        sem_wait(&writeBlock);  
  
        // Writing operation  
        data++;  
        printf("Writer writes data: %d\n", data);  
  
        sem_post(&writeBlock);i++;  
    }  
}
```

```
int main() {  
    pthread_t readers, writers;  
    sem_init(&mutex, 0, 1);  
    sem_init(&writeBlock, 0, 1); pthread_create(&readers,  
    NULL, reader, NULL);pthread_create(&writers, NULL,  
    writer, NULL); pthread_join(readers, NULL);  
    pthread_join(writers, NULL); sem_destroy(&mutex);
```

```
    sem_destroy(&writeBlock);  
    return 0;  
}
```

OUTPUT :

```
Writer writes data: 1  
Reader reads data: 1  
Writer writes data: 2  
Reader reads data: 2  
Writer writes data: 3  
Reader reads data: 3  
Writer writes data: 4  
Reader reads data: 4  
Writer writes data: 5  
Reader reads data: 5  
Writer writes data: 6  
Reader reads data: 6  
Writer writes data: 7  
Reader reads data: 7  
Writer writes data: 8  
Reader reads data: 8  
Writer writes data: 9  
Reader reads data: 9  
Writer writes data: 10  
Reader reads data: 10  
  
-----  
Process exited after 12.44 seconds with
```

21. Develop a C program to implement worst fit algorithm of memory management.

PROGRAM:

```
#include <stdio.h>

#define MAX_MEMORY 1000int

memory[MAX_MEMORY];

// Function to initialize memoryvoid
initializeMemory() {
    for (int i = 0; i < MAX_MEMORY; i++) {
        memory[i] = -1; // -1 indicates that the memory is unallocated
    }
}

// Function to display memory statusvoid
displayMemory() {
    int i, j;
    int count = 0; printf("Memory
    Status:\n");

    for (i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            count++;
            j = i;
            while (memory[j] == -1 && j < MAX_MEMORY) {j++;
            }
            printf("Free memory block %d-%d\n", i, j - 1);i = j -
            1;
        }
    }

    if (count == 0) {
        printf("No free memory available.\n");
    }
}

// Function to allocate memory using worst-fit algorithmvoid
allocateMemory(int processId, int size) {
    int start = -1;
    int blockSize = 0;

    for (int i = 0; i < MAX_MEMORY; i++) {if
    (memory[i] == -1) {
        if (blockSize == 0) {
            start = i;
        }
        blockSize++;
    }
```

```

    } else {
        blockSize = 0;
    }

    if (blockSize >= size) {
        break;
    }
}

if (blockSize >= size) {
    for (int i = start; i < start + size; i++) {
        memory[i] = processId;
    }
    printf("Allocated memory block %d-%d to Process %d\n", start, start + size - 1,
processId);
} else {
    printf("Memory allocation for Process %d failed (not enough contiguous
memory).\n", processId);
}
}

// Function to deallocate memory
void deallocateMemory(int processId) {
    for (int i = 0; i < MAX_MEMORY; i++) {if
        (memory[i] == processId) {
            memory[i] = -1;
        }
    }
    printf("Memory released by Process %d\n", processId);
}

int main() {
    initializeMemory();
    displayMemory();

    allocateMemory(1, 200);
    displayMemory();

    allocateMemory(2, 300);
    displayMemory();

    deallocateMemory(1);
    displayMemory();

    allocateMemory(3, 400);
    displayMemory();

    return 0;
}

```

}

OUTPUT:

```
Memory Management Scheme - Worst Fit
Enter the number of blocks:3
Enter the number of files:2

Enter the size of the blocks:-
Block 1:5
Block 2:2
Block 3:7
Enter the size of the files :-
File 1:1
File 2:4

File_no:      File_size :      Block_no:      Block_size:      Fragement
1             1             3             7             6
2             4             1             5             1_
```

22. Construct a C program to implement best fit algorithm of memory management.

PROGRAM:

```
#include <stdio.h>

#define MAX_MEMORY 1000int

memory[MAX_MEMORY];

// Function to initialize memoryvoid
initializeMemory() {
    for (int i = 0; i < MAX_MEMORY; i++) {
        memory[i] = -1; // -1 indicates that the memory is unallocated
    }
}

// Function to display memory statusvoid
displayMemory() {
    int i, j;
    int count = 0; printf("Memory
    Status:\n");

    for (i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            count++;
            j = i;
            while (memory[j] == -1 && j < MAX_MEMORY) {j++;
            }
            printf("Free memory block %d-%d\n", i, j - 1);i = j -
            1;
        }
    }

    if (count == 0) {
        printf("No free memory available.\n");
    }
}

// Function to allocate memory using best-fit algorithmvoid
allocateMemory(int processId, int size) {
    int start = -1;
    int blockSize = MAX_MEMORY;int
    bestStart = -1;
    int bestSize = MAX_MEMORY;

    for (int i = 0; i < MAX_MEMORY; i++) {if
        (memory[i] == -1) {
            if (blockSize == MAX_MEMORY) {start =
                i;
```

```

        }
        blockSize++;
    } else {
        if (blockSize >= size && blockSize < bestSize) {bestSize
            = blockSize;
            bestStart = start;
        }
        blockSize = 0;
    }
}

if (bestSize >= size) {
    for (int i = bestStart; i < bestStart + size; i++) {
        memory[i] = processId;
    }
    printf("Allocated memory block %d-%d to Process %d\n", bestStart,bestStart +
size - 1, processId);
} else {
    printf("Memory allocation for Process %d failed (not enough contiguous
memory).\n", processId);
}
}

// Function to deallocate memory
void deallocateMemory(int processId) {
    for (int i = 0; i < MAX_MEMORY; i++) {if
        (memory[i] == processId) {
            memory[i] = -1;
        }
    }
    printf("Memory released by Process %d\n", processId);
}

int main() {
    initializeMemory();
    displayMemory();

    allocateMemory(1, 200);
    displayMemory();

    allocateMemory(2, 300);
    displayMemory();

    deallocateMemory(1);
    displayMemory();

    allocateMemory(3, 400);
    displayMemory();

    return 0;
}

```

OUTPUT:

```
Memory Status:
Free memory block 0-999
Allocated memory block -1-198 to Process 1
Memory Status:
Free memory block 199-999
Allocated memory block -1-298 to Process 2
Memory Status:
Free memory block 299-999
Memory released by Process 1
Memory Status:
Free memory block 299-999
Allocated memory block -1-398 to Process 3
Memory Status:
Free memory block 399-999

-----
Process exited after 0.06954 seconds with return value 0
Press any key to continue . . . |
```

23. Construct a C program to implement first fit algorithm of memory management.

PROGRAM:

```
#include <stdio.h>

#define MAX_MEMORY 1000int

memory[MAX_MEMORY];

// Function to initialize memoryvoid
initializeMemory() {
    for (int i = 0; i < MAX_MEMORY; i++) {
        memory[i] = -1; // -1 indicates that the memory is unallocated
    }
}

// Function to display memory statusvoid
displayMemory() {
    int i, j;
    int count = 0; printf("Memory
Status:\n");
    for (i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            count++;
            j = i;
            while (memory[j] == -1 && j < MAX_MEMORY) {j++;
            }
            printf("Free memory block %d-%d\n", i, j - 1);i = j -
            1;
        }
    }

    if (count == 0) {
        printf("No free memory available.\n");
    }
}

// Function to allocate memory using first-fit algorithmvoid
allocateMemory(int processId, int size) {
    int start = -1;
    int blockSize = 0;

    for (int i = 0; i < MAX_MEMORY; i++) {if
    (memory[i] == -1) {
        if (blockSize == 0) {
            start = i;
        }
        blockSize++;
    } else {
```

```

        blockSize = 0;
    }

    if (blockSize >= size) {
        break;
    }
}

if (blockSize >= size) {
    for (int i = start; i < start + size; i++) {
        memory[i] = processId;
    }
    printf("Allocated memory block %d-%d to Process %d\n", start, start + size - 1,
processId);
} else {
    printf("Memory allocation for Process %d failed (not enough contiguous
memory).\n", processId);
}
}

// Function to deallocate memory
void deallocateMemory(int processId) {
    for (int i = 0; i < MAX_MEMORY; i++) {if
        (memory[i] == processId) {
            memory[i] = -1;
        }
    }
    printf("Memory released by Process %d\n", processId);
}

int main() {
    initializeMemory();
    displayMemory();

    allocateMemory(1, 200);
    displayMemory();

    allocateMemory(2, 300);
    displayMemory();

    deallocateMemory(1);
    displayMemory();

    allocateMemory(3, 400);
    displayMemory();

    return 0;
}

```

OUTPUT:

```
Memory Status:
Free memory block 0-999
Allocated memory block 0-199 to Process 1
Memory Status:
Free memory block 200-999
Allocated memory block 200-499 to Process 2
Memory Status:
Free memory block 500-999
Memory released by Process 1
Memory Status:
Free memory block 0-199
Free memory block 500-999
Allocated memory block 500-899 to Process 3
Memory Status:
Free memory block 0-199
Free memory block 900-999

-----
Process exited after 0.01792 seconds with return value 0
Press any key to continue . . . |
```

24. Design a C program to demonstrate UNIX system calls for filemanagement.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

int main() {
    int fd;
    char buffer[100];

    // Creating a new file
    fd = creat("sample.txt", S_IRWXU); if (fd
    == -1) {
        perror("create");
        exit(1);
    } else {
        printf("File 'sample.txt' created successfully.\n"); close(fd);
    }

    // Opening an existing file for writing
    fd = open("sample.txt", O_WRONLY | O_APPEND); if (fd ==
    -1) {
        perror("open");
        exit(1);
    } else {
        printf("File 'sample.txt' opened for writing.\n");
    }

    // Writing data to the file write(fd,
    "Hello, World!\n", 14);
    printf("Data written to 'sample.txt'.\n");
    close(fd);

    // Opening the file for reading
    fd = open("sample.txt", O_RDONLY); if (fd
    == -1) {
        perror("open");
        exit(1);
    } else {
        printf("File 'sample.txt' opened for reading.\n");
    }

    // Reading data from the file
    int bytesRead = read(fd, buffer, sizeof(buffer)); if
    (bytesRead == -1) {
        perror("read");
        exit(1);
    }
}
```

```
} else {  
    printf("Data read from 'sample.txt':\n"); write(STDOUT_FILENO, buffer,  
    bytesRead);  
}  
close(fd);  
  
// Deleting the file  
if (remove("sample.txt") == -1) {  
    perror("remove");  
    exit(1);  
} else {  
    printf("File 'sample.txt' deleted.\n");  
}  
}
```

```
    return 0;
}
```

OUTPUT:

```
File 'sample.txt' created successfully.
File 'sample.txt' opened for writing.
Data written to 'sample.txt'.
File 'sample.txt' opened for reading.
Data read from 'sample.txt':
Hello, World!
File 'sample.txt' deleted.
```

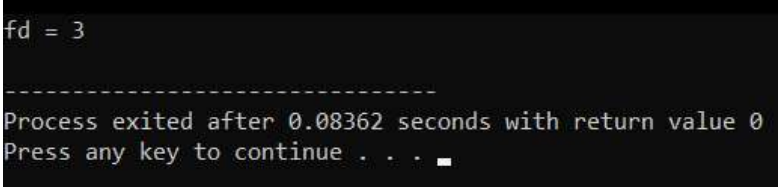
```
-----
Process exited after 0.02066 seconds with return value 0
Press any key to continue . . . |
```

25) Construct a C program to implement the I/O system calls of UNIX (fcntl, seek, stat, opendir, readdir)

PROGRAM:

```
#include<stdio.h>
#include<fcntl.h>
#include<errno.h>
extern int errno; int
main()
{
    int fd = open("foo.txt", O_RDONLY | O_CREAT);
    printf("fd = %d\n", fd);
    if (fd == -1)
    {
        printf("Error Number % d\n", errno);
        perror("Program");
    }
    return 0;
}
```

OUTPUT:

A terminal window with a black background and white text. The first line shows 'fd = 3'. Below it is a dashed line. The next line says 'Process exited after 0.08362 seconds with return value 0'. The final line says 'Press any key to continue . . . ' followed by a small white square cursor.

```
fd = 3
-----
Process exited after 0.08362 seconds with return value 0
Press any key to continue . . . ■
```

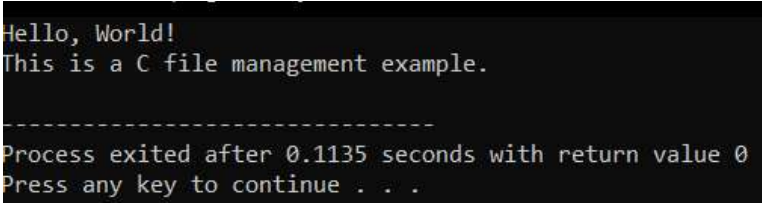
26) Construct a C program to implement the file management operations.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int
main() {
    FILE *file;
    file = fopen("example.txt", "w");
    if (file == NULL) {
        printf("Error opening the file for writing.\n");
        return 1;
    }
    fprintf(file, "Hello, World!\n");
    fprintf(file, "This is a C file management example.\n");
    fclose(file);
    file = fopen("example.txt", "r");
    if (file == NULL) {
        printf("Error opening the file for reading.\n");
        return 1;
    }
    char buffer[100];
    while (fgets(buffer, sizeof(buffer), file) != NULL) {
        printf("%s", buffer);
    }
    fclose(file);

    return 0;
}
```

OUTPUT:



```
Hello, World!
This is a C file management example.
-----
Process exited after 0.1135 seconds with return value 0
Press any key to continue . . .
```

27) Develop a C program for simulating the function of ls UNIXCommand.

PROGRAM:

```
#include<stdio.h>
#include<dirent.h>
int main()
{
    char fn[10], pat[10], temp[200];FILE
    *fp;
    printf("\n Enter file name : ");
    scanf("%s", fn);
    printf("Enter the pattern: ");
    scanf("%s", pat);
    fp = fopen(fn, "r");
    while (!feof(fp)) { fgets(temp,
        sizeof(fp), fp);if (strcmp(temp,
        pat))
        printf("%s", temp);
    }
    fclose(fp);
    return 1;
}
```

OUTPUT:

```
This is a sample line.
Hello, World!
Sample pattern in this line.
Another sample line.
```

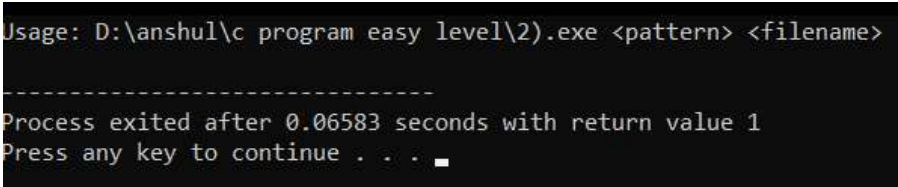
28) Write a C program for simulation of GREP UNIX command.

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LINE_LENGTH 1024
void searchFile(const char *pattern, const char *filename)
{
    FILE *file = fopen(filename, "r");
    if (file == NULL) { perror("Error
        opening file");exit(1);
    }
    char line[MAX_LINE_LENGTH]; while
    (fgets(line, sizeof(line), file)) {
        if (strstr(line, pattern) != NULL) {
            printf("%s", line);
        }
    }
    fclose(file);
}

int main(int argc, char *argv[]) {if
    (argc != 3) {
        fprintf(stderr, "Usage: %s <pattern> <filename>\n", argv[0]);return 1;
    }
    const char *pattern = argv[1]; const
    char *filename = argv[2];
    searchFile(pattern, filename);
    return 0;
}
```

OUTPUT:



```
Usage: D:\anshul\c program easy level\2).exe <pattern> <filename>
-----
Process exited after 0.06583 seconds with return value 1
Press any key to continue . . . ■
```

29) Write a C program to simulate the solution of Classical Process Synchronization Problem

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int
mutex = 1;
int full = 0;
int empty = 10, x = 0;
void producer()
{
    --mutex;
    ++full;
    --empty;
    x++;
    printf("\nProducer produces"
           "item %d",
           x);
    ++mutex;
}
void consumer()
{
    --mutex;
    --full;
    ++empty;
    printf("\nConsumer consumes ""item
           %d",
           x);
    x--;
    ++mutex;
}
int main()
{
    int n, i;
    printf("\n1. Press 1 for Producer"
```

```

        "\n2. Press 2 for Consumer""\n3.
        Press 3 for Exit");
#pragma omp critical for (i
    = 1; i > 0; i++)
    {
        printf("\nEnter your choice:");
        scanf("%d", &n);
        switch (n) {
        case 1:
            if ((mutex == 1)
                && (empty != 0)) {
                producer();
            }
            else
                {
                    printf("Buffer is full!");
                }
            break;
        case 2:
            if ((mutex == 1) &&
                (full != 0)) {
                consumer();
            }
            else {
                printf("Buffer is empty!");
            }
            break;

        case 3:
            exit(0);
            break;
        }
    }
}

```

OUTPUT:

```
1. Press 1 for Producer
2. Press 2 for Consumer
3. Press 3 for Exit
Enter your choice:1
```

```
Producer produces item 1
Enter your choice:1
```

```
Producer produces item 2
Enter your choice:1
```

```
Producer produces item 3
Enter your choice:2
```

```
Consumer consumes item 3
Enter your choice:3
```

```
-----
Process exited after 6.797 seconds with return value 0
Press any key to continue . . .
```

30. Write C programs to demonstrate the following thread related concepts.

PROGRAM:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
void* func(void* arg)
{
    pthread_detach(pthread_self());printf("Inside
    the thread\n"); pthread_exit(NULL);
}
void fun()
{
    pthread_t ptid;
    pthread_create(&ptid, NULL, &func, NULL);
    printf("This line may be printed"
        " before thread terminates\n");
    if(pthread_equal(ptid, pthread_self()))
    {
        printf("Threads are equal\n");
    }

    else
        printf("Threads are not equal\n");
    pthread_join(ptid, NULL);
    printf("This line will be printed""
        after thread ends\n");
    pthread_exit(NULL);
}

int main()
{
    fun();
    return 0;
}
```

OUTPUT:

```
This line may be printed before thread terminates
Inside the thread
Threads are not equal
This line will be printed after thread ends
```

31. Construct a C program to simulate the First in First Out paging technique of memory management.

AIM: Construct a C program to simulate the First in First Out paging technique of memory management.

ALGORITHM:

1. Create an array to represent the page frames in memory.
2. Initialize all page frames to -1, indicating that they are empty.
3. Initialize a queue to keep track of the order in which pages are loaded into memory.
4. Initialize variables for page hits and page faults to zero.
5. Read the reference string (sequence of page numbers) from the user or use a predefined array.
6. For each page in the reference string, do the following:
7. Check if the page is already in memory (a page hit).
If it's a page hit, update the display and move to the next page.
8. If it's a page fault (page not in memory), do the following:
Increment the page fault count.
9. Remove the oldest page in memory (the one at the front of the queue).
10. Load the new page into the memory and enqueue it.
11. Update the display to show the page replacement.
12. Continue this process for all pages in the reference string.
13. After processing all pages, display the total number of page faults.

PROGRAM:

```
#include <stdio.h>

#define MAX_FRAMES 3 // Maximum number of frames in memory

void printFrames(int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) {
            printf(" - ");
        } else {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}

int main() {
    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2}; int n =
```

```

sizeof(referenceString) / sizeof(referenceString[0]);
int frames[MAX_FRAMES];

int framePointer = 0; // Points to the current frame to be replaced

for (int i = 0; i < MAX_FRAMES; i++)
{
    frames[i] = -1; // Initialize all frames to -1 (indicating empty)
}

printf("Reference String: ");for
(int i = 0; i < n; i++) {
    printf("%d ", referenceString[i]);
}printf("\n\n");
printf("Page Replacement Order:\n");
for (int i = 0; i < n; i++) {
    int page = referenceString[i];int
    pageFound = 0;
    // Check if the page is already in memoryfor (int j
    = 0; j < MAX_FRAMES; j++) {
        if (frames[j] == page) {
            pageFound = 1; break;
        }
    }
}

    if (!pageFound) {
        printf("Page %d -> ", page);
        frames[framePointer] = page;
        framePointer = (framePointer + 1) % MAX_FRAMES;
        printFrames(frames, MAX_FRAMES);
    }
}

return 0;
}

```

OUTPUT:

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:

Page 7 -> 7 - -

Page 0 -> 7 0 -

Page 1 -> 7 0 1

Page 2 -> 2 0 1

Page 3 -> 2 3 1

Page 0 -> 2 3 0

Page 4 -> 4 3 0

Page 2 -> 4 2 0

Page 3 -> 4 2 3

Page 0 -> 0 2 3

Process exited after 0.0623 seconds with return value 0

Press any key to continue . . . |

32. Construct a C program to simulate the Least Recently Used paging technique of memory management.

AIM: Construct a C program to simulate the Least Recently Used paging technique of memory management.

ALGORITHM:

1. Create an array to represent the page frames in memory.
2. Initialize all page frames to -1, indicating that they are empty.
3. Create a queue or a data structure (e.g., a doubly-linked list) to maintain the order of pages based on their usage history.
4. Initialize a counter for page hits and page faults to zero.
5. Read the reference string (sequence of page numbers) from the user or use a predefined array.
6. For each page in the reference string, do the following:
 7. Check if the page is already in memory (a page hit).
 8. If it's a page hit, update the position of the page in the usage history data structure to indicate it was recently used.
 9. If it's a page fault (page not in memory), do the following:
 10. Increment the page fault count.
 11. Find the least recently used page in the usage history data structure (e.g., the front of the queue or the tail of the list).
 12. Remove the least recently used page from memory and the usage history.
 13. Load the new page into memory and add it to the back of the usage history.
 14. Update the display to show the page replacement.
 15. Continue this process for all pages in the reference string.
15. After processing all pages, display the total number of page faults.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_FRAMES 3

void printFrames(int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) {
            printf(" - ");
        } else {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}
```

```

int main() {
    int frames[MAX_FRAMES];
    int usageHistory[MAX_FRAMES]; // To store the usage history of pages
    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1; // Initialize frames to -1 (empty)
        usageHistory[i] = 0; // Initialize usage history
    }

    int pageFaults = 0;
    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2}; int n =
    sizeof(referenceString) / sizeof(referenceString[0]);

    printf("Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", referenceString[i]);
    }
    printf("\n\n");

    printf("Page Replacement Order:\n");

    for (int i = 0; i < n; i++) {
        int page = referenceString[i];
        int pageFound = 0;

        // Check if the page is already in memory (a page hit)
        for (int j = 0; j < MAX_FRAMES; j++) {
            if (frames[j] == page) {
                pageFound = 1;
                // Update the usage history by incrementing other pages
                for (int k = 0; k < MAX_FRAMES; k++) {
                    if (k != j) {
                        usageHistory[k]++;
                    }
                }
                usageHistory[j] = 0; // Reset the usage counter for the used page
            }
        }

        if (!pageFound) {

```

```

    printf("Page %d -> ", page);

    // Find the page with the maximum usage counter (least recently
used)
    int lruPage = 0;
    for (int j = 1; j < MAX_FRAMES; j++) {
        if (usageHistory[j] > usageHistory[lruPage]) {lruPage
            = j;
        }
    }

    int replacedPage = frames[lruPage];
    frames[lruPage] = page;
    usageHistory[lruPage] = 0;

    if (replacedPage != -1) {
        printf("Replace %d with %d: ", replacedPage, page);
    } else {
        printf("Load into an empty frame: ");
    }

    printFrames(frames, MAX_FRAMES);
    pageFaults++;
}
}

printf("\nTotal Page Faults: %d\n", pageFaults);

return 0;
}

```

OUTPUT:

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:

Page 7 -> Load into an empty frame: 7 - -

Page 0 -> Replace 7 with 0: 0 - -

Page 1 -> Replace 0 with 1: 1 - -

Page 2 -> Replace 1 with 2: 2 - -

Page 0 -> Replace 2 with 0: 0 - -

Page 3 -> Replace 0 with 3: 3 - -

Page 0 -> Replace 3 with 0: 0 - -

Page 4 -> Replace 0 with 4: 4 - -

Page 2 -> Replace 4 with 2: 2 - -

Page 3 -> Replace 2 with 3: 3 - -

Page 0 -> Replace 3 with 0: 0 - -

Page 3 -> Replace 0 with 3: 3 - -

Page 2 -> Replace 3 with 2: 2 - -

Total Page Faults: 13

Process exited after 0.05045 seconds with return value 0

Press any key to continue . . . |

33. Construct a C program to simulate the optimal paging technique of memory management

AIM: Construct a C program to simulate the optimal paging technique of memory management

ALGORITHM:

1. Create an array to represent the page frames in memory.
2. Initialize all page frames to -1, indicating that they are empty.
3. Initialize a variable for page faults to zero.
4. Read the reference string (sequence of page numbers) from the user or use a predefined array.
5. For each page in the reference string, do the following:
6. Check if the page is already in memory (a page hit).
7. If it's a page hit, move to the next page.
8. If it's a page fault (page not in memory), do the following:
9. Increment the page fault count.
10. Calculate the future references of each page in memory by scanning the remaining part of the reference string.
11. Find the page that will not be used for the longest time in the future (the optimal page to replace).
12. Replace the optimal page with the new page.
13. Continue this process for all pages in the reference string.
13. After processing all pages, display the total number of page faults.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_FRAMES 3

void printFrames(int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) {
            printf(" - ");
        } else {
            printf(" %d ", frames[i]);
        }
    }
    printf("\n");
}

int main() {
    int frames[MAX_FRAMES];
    for (int i = 0; i < MAX_FRAMES; i++) { frames[i] = -1;
```

```

    // Initialize frames to -1 (empty)
}

int pageFaults = 0;
int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2}; int n =
sizeof(referenceString) / sizeof(referenceString[0]);

printf("Reference String: ");for
(int i = 0; i < n; i++) {
    printf("%d ", referenceString[i]);
}
printf("\n\n");

printf("Page Replacement Order:\n");for

(int i = 0; i < n; i++) {
    int page = referenceString[i];
    int pageFound = 0;

    // Check if the page is already in memory (a page hit)for (int j
= 0; j < MAX_FRAMES; j++) {
        if (frames[j] == page) {
            pageFound = 1; break;
        }
    }

    if (!pageFound) {
        printf("Page %d -> ", page);

        int optimalPage = -1; int
farthestDistance = 0;

        for (int j = 0; j < MAX_FRAMES; j++) {int
futureDistance = 0;
            for (int k = i + 1; k < n; k++) {
                if (referenceString[k] == frames[j]) {break;
                }
                futureDistance++;
            }

            if (futureDistance > farthestDistance) {
                farthestDistance = futureDistance;
                optimalPage = j;
            }
        }
    }
}

```

```

        frames[optimalPage] = page;

        printFrames(frames, MAX_FRAMES);
        pageFaults++;
    }
}

printf("\nTotal Page Faults: %d\n", pageFaults);

return 0;
}

```

OUTPUT

```

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:
Page 7 -> 7 - -
Page 0 -> 0 - -
Page 1 -> 0 1 -
Page 2 -> 0 2 -
Page 3 -> 0 2 3
Page 4 -> 4 2 3
Page 0 -> 0 2 3

Total Page Faults: 7

-----
Process exited after 0.05286 seconds with return value 0
Press any key to continue . . . |

```